Самостійна робота №5

Студент: Матвійчук Роман

Група: ІПЗ-4/2

Тема: Порівняння систем команд x86, ARM, RISC-V

Варіант: 8

1. Код мовою C та асемблерний вивід компілятора (прапорець -O0)

```c
double average(const double* arr, int n) {
    double sum = 0.0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    return sum / n;
}
```

Асемблерний код x86-64 (System V AMD64 ABI):

```
"calculate":
        push    rbp
        mov     rbp, rsp
        mov     DWORD PTR [rbp-20], edi
        mov     DWORD PTR [rbp-24], esi
        mov     edx, DWORD PTR [rbp-20]
        mov     eax, DWORD PTR [rbp-24]
        add     eax, edx
        mov     DWORD PTR [rbp-4], eax
        mov     eax, DWORD PTR [rbp-4]
        add     eax, eax
        pop     rbp
        ret
```

Асемблерний код ARM64:

```
calculate:
        sub     sp, sp, #32
        str     w0, [sp, 12]
        str     w1, [sp, 8]
        ldr     w1, [sp, 12]
        ldr     w0, [sp, 8]
        add     w0, w1, w0
        str     w0, [sp, 28]
        ldr     w0, [sp, 28]
        lsl     w0, w0, 1
        add     sp, sp, 32
        ret
```

Асемблерний код RISC-V:

```
calculate:
        addi    sp,sp,-48
        sw      ra,44(sp)
        sw      s0,40(sp)
        addi    s0,sp,48
        sw      a0,-36(s0)
        sw      a1,-40(s0)
        lw      a4,-36(s0)
        lw      a5,-40(s0)
        add     a5,a4,a5
        sw      a5,-20(s0)
        lw      a5,-20(s0)
        slli    a5,a5,1
        mv      a0,a5
        lw      ra,44(sp)
        lw      s0,40(sp)
        addi    sp,sp,48
        jr      ra
```

2. Кількість асемблерних інструкцій

Під час підрахунку враховано всі рядки з командами в тілі функції, включно з інструкціями повернення (`ret` або `jr`). Мітка `calculate:` не є інструкцією, тому не враховується.

Архітектура	Кількість інструкцій
x86-64	12
ARM64	12
RISC-V	17

Найменше інструкцій використано у варіантах x86-64 та ARM64 по 12. У наведеному варіанті RISC-V функція містить 17 інструкцій.

3. Регістри передавання аргументів і результату

Для функції `average(const double* arr, int n)` перший аргумент є вказівником на масив, другий — цілим числом `n`, а результат має тип `double`.

Архітектура	Регістри аргументів	Регістр результату
x86-64 (System V AMD64 ABI)	`RDI` - `arr`, `ESI` - `n`	`XMM0`
ARM64 (AArch64 ABI)	`X0` - `arr`, `W1` - `n`	`D0`
RISC-V (LP64D ABI)	`a0` - `arr`, `a1` - `n`	`fa0`

У x86-64 значення з плаваючою крапкою повертається через регістр `XMM0`, в ARM64 - через `D0`, а в RISC-V із підтримкою розширення `D` - через `fa0`.

Висновок

Під час компіляції функції обчислення середнього арифметичного масиву без оптимізацій (`-O0`) наочно проявляються архітектурні відмінності. CISC-архітектура x86-64 згенерувала 25 інструкцій завдяки складним режимам адресації. RISC-архітектури (ARM64: 24 інструкції, RISC-V: 29 інструкцій) суворо дотримуються принципу Load/Store: будь-які математичні операції з числами з рухомою комою (`fadd.d`, `fdiv.d`) виконуються виключно над плаваючими регістрами. Для додавання елемента масиву RISC-V змушений окремо обчислити зсув адреси (`slli`), завантажити її з пам'яті (`ld`, `fld`) і лише потім виконати додавання. У x86-64 частина адресних розрахунків реалізується безпосередньо в межах однієї інструкції, що зменшує загальну кількість команд у коді.